

Flutter Development Standards

Logistics Reverse Auction & Transporter Bidding Platform

Prepared by: Anshuman Tiwari **Purpose:** These are the standards every screen, widget, and service in the Flutter app must follow. Give this document to your LLM alongside the PRD, Tech Stack, and Database Schema documents so generated code is consistent across the whole app instead of every screen looking hand-rolled differently.

1. Project Structure

Keep it flat and predictable — no over-engineered clean architecture for this timeline, but still organized enough that any file's purpose is obvious at a glance.

```
lib/
├─ main.dart
├─ config/
│   ├── app_theme.dart
│   ├── app_routes.dart
│   └─ api_constants.dart
├─ models/
│   ├── user_model.dart
│   ├── tender_model.dart
│   ├── bid_model.dart
│   ├── shortlist_model.dart
│   ├── negotiation_model.dart
│   ├── competitive_statement_model.dart
│   └─ notification_model.dart
├─ services/
│   ├── api_client.dart          # base dio/http setup + JWT interceptor
│   ├── auth_service.dart
│   ├── tender_service.dart
│   ├── bid_service.dart
│   ├── negotiation_service.dart
│   ├── notification_service.dart
│   └─ fcm_service.dart
├─ providers/
│   ├── auth_provider.dart
│   ├── tender_provider.dart
│   ├── bid_provider.dart
│   └─ notification_provider.dart
├─ screens/
│   ├── auth/
│   │   ├── login_screen.dart
│   │   └─ signup_screen.dart
│   ├── company/
│   │   ├── company_dashboard_screen.dart
│   │   ├── create_tender_screen.dart
│   │   ├── tender_detail_screen.dart
│   │   ├── competitive_statement_screen.dart
│   │   └─ finalize_screen.dart
│   ├── transporter/
│   │   ├── transporter_dashboard_screen.dart
│   │   ├── browse_tenders_screen.dart
│   │   ├── bid_screen.dart
│   │   └─ negotiation_screen.dart
│   └─ shared/
│       ├── notification_center_screen.dart
│       └─ profile_screen.dart
└─ widgets/
```

```

├─ primary_button.dart
├─ loading_indicator.dart
├─ error_view.dart
├─ tender_card.dart
├─ bid_leaderboard_tile.dart
└─ countdown_timer.dart

```

Rule: No widget file should contain both a `Model` class and a `Screen` class. Models live only in `models/`.

2. Naming Conventions

Item	Convention	Example
Files	<code>snake_case.dart</code>	<code>tender_detail_screen.dart</code>
Classes	<code>UpperCamelCase</code>	<code>TenderDetailScreen</code>
Variables/functions	<code>lowerCamelCase</code>	<code>fetchActiveTenders()</code>
Constants	<code>lowerCamelCase</code> with <code>k</code> prefix optional, or <code>SCREAMING_SNAKE_CASE</code> for top-level API constants	<code>kDefaultPadding</code> , <code>API_BASE_URL</code>
Private members	prefix with <code>_</code>	<code>_currentBidPrice</code>
Screens	suffix with <code>Screen</code>	<code>LoginScreen</code>
Providers	suffix with <code>Provider</code>	<code>TenderProvider</code>
Services	suffix with <code>Service</code>	<code>BidService</code>
Models	suffix with <code>Model</code>	<code>TenderModel</code>

3. State Management (Provider)

- One `Provider`/`ChangeNotifier` per major feature domain — `AuthProvider`, `TenderProvider`, `BidProvider`, `NotificationProvider`. Do not create a single giant "AppState" provider.
- Providers hold **UI state + orchestration only** — never put raw `http`/`dio` calls directly inside a provider. Providers call a `Service`, the `Service` calls the API.
- Every async provider method must expose a loading state so screens can show spinners:

```
dart
```

```

bool _isLoading = false;
bool get isLoading => _isLoading;

Future<void> fetchTenders() async {
  _isLoading = true;
  notifyListeners();
  try {
    _tenders = await _tenderService.getTenders();
  } catch (e) {
    _errorMessage = e.toString();
  } finally {
    _isLoading = false;
    notifyListeners();
  }
}

```

- Always call `notifyListeners()` after state changes — a common LLM-generated bug is forgetting this after an error branch.

4. API Service Layer

- One `ApiClient` wraps `dio` (preferred) with:
 - Base URL from `api_constants.dart`
 - JWT interceptor that attaches `Authorization: Bearer <token>` from `shared_preferences` on every request
 - A response interceptor that catches 401s and triggers logout/redirect to login
- One `Service` class per resource (`TenderService`, `BidService`, etc.) — each method returns a typed model (or `List<Model>`), never a raw `Map`/`Response`.
- **Never parse JSON inline inside a screen or provider** — parsing belongs in the model's `fromJson` factory constructor.
- Every model must implement both `fromJson` and `toJson`:

dart

```

class TenderModel {
  final int id;
  final String origin;
  final String destination;
  // ...

  factory TenderModel.fromJson(Map<String, dynamic> json) => TenderModel(
    id: json['id'],
    origin: json['origin'],
    destination: json['destination'],
  );

  Map<String, dynamic> toJson() => {
    'origin': origin,
    'destination': destination,
  };
}

```

5. Error Handling

- Every API call must be wrapped in `try/catch`. Never let an unhandled exception crash a screen.
- User-facing errors go through a single reusable `ErrorView` widget (icon + message + retry button) — don't hand-roll error UI per screen.
- Network/API errors should surface a readable message, not a raw stack trace — normalize backend error responses in `ApiClient`:

dart

```

catch (e) {
  throw Exception('Unable to load tenders. Please check your connection.');
```

- Form validation errors show inline under the relevant field, not as a generic snackbar, wherever a form exists (tender creation, bid placement).

6. UI/Widget Standards

- **Reuse before creating:** before writing a new button/card/loading widget, check `widgets/` for an existing one.
- Every screen must handle three states explicitly: **loading**, **error**, **empty** (e.g., "No tenders yet") — not just the happy path.
- Use `const` constructors wherever possible for performance and to avoid unnecessary rebuilds.

- No magic numbers for spacing/sizing — define constants in `app_theme.dart` (`kDefaultPadding = 16.0`, etc.) and reuse them.
 - Keep `build()` methods readable — extract any widget subtree used more than once, or longer than ~40 lines, into its own private `_buildX()` method or separate widget file.
 - Consistent color/typography via a single `ThemeData` in `app_theme.dart` — no inline `Color(0xFF...)` scattered across screens.
-

7. Async & Real-Time Updates (Polling — MVP approach)

Since MVP uses polling instead of WebSockets (see Tech Stack doc):

- Use a `Timer.periodic` (e.g., every 5 seconds) on the tender detail/leaderboard screen only, and **cancel it in** `dispose()` — a very common leak if left running after navigating away.

```
dart

@override
void dispose() {
  _pollTimer?.cancel();
  super.dispose();
}
```

- Do not poll from the dashboard/list screens — only from the specific detail screen the user is actively viewing.
-

8. Notifications

- FCM foreground messages must be handled and shown via `flutter_local_notifications` (FCM does not auto-display foreground notifications on Android).
 - Every notification tap should deep-link to the relevant screen (e.g., tapping "Shortlisted" opens that tender's detail screen) — do not just show the notification and leave the user on the current screen.
 - The in-app Notification Center screen is the source of truth — pull from `GET /notifications`, not from a locally cached list that can drift from the backend.
-

9. Security Basics

- **No API keys, base URLs pointing to production secrets, or Firebase service account files committed to Git.** `google-services.json` is fine to commit (it's client-safe), but never commit backend `.env`/credentials files into the Flutter repo either.
- JWT token stored via `shared_preferences` is acceptable for MVP; do not store it in a global mutable variable that could leak across sessions.
- Always log out cleanly: clear the stored JWT and FCM token association on logout.

10. Code Style & Linting

- Enable and follow `flutter_lints` (included by default in new Flutter projects) — do not disable lint rules to save time; fix the warning instead, it's usually faster than it looks.
 - Run `flutter analyze` before considering any module "done" — zero warnings is the bar, not "it compiles."
 - Format with `dart format .` before every commit.
-

11. Git Commit Standards

Use a short, meaningful prefix per commit — this also makes it easy to show a clean progress history if your recruiter looks at the repo:

Prefix	Use for
<code>feat:</code>	New feature/screen
<code>fix:</code>	Bug fix
<code>refactor:</code>	Code restructuring, no behavior change
<code>style:</code>	Formatting/UI-only tweaks
<code>docs:</code>	Documentation updates

Example: `feat: add bid placement screen with countdown timer`

Commit after each working milestone (per your build plan: setup, auth, tender CRUD, bidding, negotiation, finalization, notifications) — not one giant commit at the end.

12. What NOT to Do (Common LLM-Generated Code Pitfalls)

- Don't let the LLM generate `StatefulWidget`s that call `setState()` for data that should live in a `Provider` — this causes state to silently desync between screens (e.g., bid count not updating on dashboard after placing a bid elsewhere).
 - Don't accept hardcoded `http://10.0.2.2:8080` scattered across multiple files — it belongs once in `api_constants.dart`.
 - Don't accept business logic (e.g., "is this the L1 bid?") duplicated in the Flutter app — that calculation belongs on the backend; Flutter only displays what the API returns.
 - Don't accept a `Timer.periodic` without a matching `dispose()` cancellation — check every screen that polls.
 - Don't accept silently swallowed exceptions (`catch (e) {}` with no handling) — every catch block must either show an error or log it meaningfully.
-

13. Minimal Testing Expectation (MVP)

Full test coverage isn't realistic for this timeline, but at minimum:

- Manually test each of the 3 states (loading/error/empty) per screen before marking it done.
- Run the full end-to-end flow (per your build plan's Sunday checklist) at least twice before the demo.
- Widget/unit tests are a stretch goal only — skip for MVP unless you finish early.